ELE00146M Workspace: Image Filtering & SPI System

Project Overview This workspace contains a mixed-signal embedded system design targeting the **Digilent ZedBoard**. The system performs hardware-accelerated image filtering on the Zynq FPGA and communicates with an external microcontroller via SPI to display status or results on an LED Matrix.

Repository Structure

The project is organized into `script` folders containing source code and build/toolchain folders.

```
ELE00146M-WORKSPACE/
├── script/
│   ├── keil/             # External MCU Firmware (SPI Slave & LED Matrix)
│   ├── vitis/            # Zynq PS Embedded Software (Filter Logic & SPI Master)
│   ├── vivado/           # FPGA Hardware Design (Constraints & IP)
│   └── utils/            # Python Host Utilities (Image Conversion & Serial)
├── keil_final/           # Keil Project Build Directory
├── vitis/                # Vitis IDE Workspace
└── vivado/               # Vivado Project Directory
```

System Modules

1. FPGA Hardware (Vivado)

- **Location:** `script/vivado/`
- **Key Files:**
  - `zedboard_constraints.xdc` : Maps system signals to ZedBoard physical pins (Pmod/LEDs).
  - `Block Diagram.png` : Visual reference of the IP Integrator design.
  - `IP/` : Contains custom hardware IP blocks used in the design. Mainly, VHDL scripts for Custom IP block.
- **Role:** Defines the hardware platform, custom filter IP, and pinout configuration.

2. Embedded Processing (Vitis)

- **Location:** `script/vitis/`
- **Key Files:**
  - `platform_filter_spi_03.xsa` : Hardware Handoff file exported from Vivado.
  - `main_filter_test_1.c` : Main script for testing Custom IP Block that perform exactly the same behavier as TESTBENCH in vivado.
  - `main_filter_test_2.c` : Main script that perform image filter with customable kernel to lenna image and Stream data out to Serial via python script. (PART A)
  - `main_filter_spi_test.c` : Main script for sending filtered alphabet image to LED Matrix. (PART B)
  - `main_spi_test.c` : Main application for sending data to STM32 via SPI.
  - `myfilter_utils.h` : Driver functions for the custom hardware filter IP.
  - `spi_led.h` : SPI Master driver to communicate with the external MCU.
  - `image_data_lenna.h` : C-header file containing the test lenna image data.
  - `image_data.h` : C-header file containing the test alpahbet image data.

3. External Microcontroller (Keil)

- **Location:** `script/keil/`
- **Key Files:**
  - `main.c` : Main control loop for the external MCU.
  - `spi_slave.h` : Implements SPI Slave logic to receive data from the ZedBoard.
  - `matrixLED.c` : Drivers to drive the connected LED Matrix display.
  - `matrixLED.h` : Headder for LED Matrix display's Driver.
- **Role:** Acts as an SPI slave peripheral, receiving processed data or status from the FPGA and displaying it on an LED Matrix.

4. Host Utilities (Python)

- **Location:** `script/utils/`
- **Key Files:**
  - `image_to_header.py` : Converts standard image files (PNG/JPG) into C arrays (`.h`) for use in Vitis. (PART A/B)
  - `serial_image.py` : Listens on UART for processed image data sent back to the PC. (PART A) Using with `script/vitis/main_filter_test_2.c` .
  - `environment.yml` : Conda environment configuration.

Xilinx Quad SPI Master <-> STM32 8x8 LED Matrix Slave

Signal	Zedboard Pin	Port	STM32 Pin
CS (SS)	W12	JB1	PB12 (NSS)
MOSI (IO0)	W11	JB2	PB15 (MOSI)
MISO (IO1)	V10	JB3	PB14 (MISO)
SCLK	W8	JB4	PB13 (SCK)
GND	GND	GND	GND

Getting Started

Step 1: Python Environment Setup

Install the necessary Python dependencies for the utility scripts.

```
cd script/utils
# If using Conda
conda env create -f environment.yml
# Or manually
pip install pyserial numpy pillow
```

Step 2: Prepare Image Data

If you wish to change the input image:

```
python img_to_header.py <path_to_image>
```

Move the generated `image_data.h` to the `vitis` project directory.

Step 3: Vivado Hardware Build

1. Open Vivado and navigate to the `vivado` directory.
2. Ensure `zedboard_constraints.xdc` is active.
3. Generate the Bitstream.
4. **File** -> Export -> **Export Hardware** (Include Bitstream) to update the `.xsa` file if the hardware has changed.

Step 4: Vitis Software Deployment

1. Launch Vitis.
2. Create a **Platform Project** using `platform_filter_spi_03.xsa` .
3. Create an **Application Project** targeting that platform.
4. Import sources from `script/vitis/` (specifically `main_filter_test_2.c` and headers).
5. Build the project and **Run as -> Launch on Hardware**.
6. Run python script to recieve serial to image `python serial_image.py` at `script/utils/`

Step 5: External MCU Setup

1. Open the project in Keil uVision (`keil_final` or import script from `script/keil`).
2. Compile and Flash the firmware to the external microcontroller.
3. **Physical Connection:** Connect the SPI pins (MISO, MOSI, CLK, CS) and GND between the ZedBoard Pmod header and the External MCU as defined in `zedboard_constraints.xdc` .